

SQL stored procedures - cursor

LEERDOELEN

1. Je kan een cursor implementeren.

VOORBEELDOPLOSSINGEN OEFENINGEN

1. Schrijf de SQL-instructie die de SQL stored procedure met naam "SP_ BESTPLAYERS" aanmaakt. De SQL stored procedure moet in onderstaande functionaliteit voorzien.

We wensen de top ... van spelers die het meeste wedstrijden hebben gewonnen op te vragen. Op basis van de waarde van een parameter wordt bepaald hoeveel spelers. Bv 5 om de top 5 op te vragen.

Als het gevraagde aantal hoger ligt dan het totaal aantal spelers in de met gewonnen wedstrijden stop je met de verwerking wanneer je de laatste speler hebt verwerkt.

Je toont voor elke speler een volgnummer dat de positie aangeeft, de spelersnummer en naam van de speler en een telling van het aantal gewonnen wedstrijden.

Hou rekening met volgende technische vereisten:

- Je moet gebruik maken van een cursor.
- Je geeft de top ... terug als een resultaatset.
- Je mag GEEN gebruik maken van de LIMIT () en COUNT() functie!

```

CREATE OR ALTER PROCEDURE [dbo].[SP_BESTPLAYERS]
(
    @prm_numberofrows INT
)
AS
BEGIN
    DECLARE @var_playernr INT;
    DECLARE @var_currentplayernr INT;
    DECLARE @var_gameswon INT;
    DECLARE @var_sortnumber INT;

    -- Create temporary table
    CREATE TABLE #tempScores (PLAYERNR INT, GAMESWON INT, RANKING INT);
    -- Initialize variables
    SET @var_playernr = -1;
    SET @var_gameswon = 1;

    -- Declare and open cursor and read first record
    DECLARE var_cursor CURSOR FOR SELECT SPELERSNR FROM [dbo].[WEDSTRIJDEN] WHERE [GEWONNEN] > [VERLOREN] ORDER BY [SPELERSNR];
    OPEN var_cursor;
    FETCH NEXT FROM var_cursor INTO @var_playernr;
    SET @var_currentplayernr = @var_playernr;

    -- Process cursor
    WHILE @@FETCH_STATUS = 0
    BEGIN
        -- Read next record
        FETCH NEXT FROM var_cursor INTO @var_playernr;

        IF (@var_playernr <> @var_currentplayernr)
        BEGIN
            -- Insert record in temp table if player read differs from previous player
            INSERT INTO #tempScores VALUES (@var_currentplayernr, @var_gameswon, 0);
            -- Change/reset variables
            SET @var_currentplayernr = @var_playernr;
            SET @var_gameswon = 1;
        END
        ELSE
            SET @var_gameswon = @var_gameswon + 1;
    END

    -- Dispose of first cursor
    CLOSE var_cursor;
    DEALLOCATE var_cursor;

    -- Declare next cursor for setting ranking
    DECLARE var_cursorsort CURSOR FOR SELECT [PLAYERNR] FROM #tempScores ORDER BY [GAMESWON] DESC;
    OPEN var_cursorsort;
    FETCH NEXT FROM var_cursorsort INTO @var_playernr;
    SET @var_sortnumber = 1;

    WHILE @@FETCH_STATUS = 0
    BEGIN
        -- Set ranking for player of current record
        UPDATE [#tempScores] SET
            [RANKING] = @var_sortnumber
        WHERE
            [PLAYERNR] = @var_playernr;

        -- Read next record
        FETCH NEXT FROM var_cursorsort INTO @var_playernr;
        SET @var_sortnumber = @var_sortnumber + 1;
    END

    -- Dispose of second cursor
    CLOSE var_cursorsort;
    DEALLOCATE var_cursorsort;

    -- Delete all records from temp table where ranking is higher then number of rows requested
    DELETE FROM [#tempScores] WHERE [RANKING] > @prm_numberofrows;
    -- Get requested information
    SELECT
        [t].[RANKING],
        [t].[PLAYERNR],
        [s].[NAAM],
        [t].[GAMESWON]
    FROM
        [#tempScores] [t]
        INNER JOIN [dbo].[SPELERS] [s] ON [t].[PLAYERNR] = [s].[SPELERSNR]
    ORDER BY [t].[RANKING];
END
GO

```